
TRÍ TUỆ NHÂN TẠO

Machine Learning

Phần 6 — Clustering Algorithms

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	4
1 Thuật toán phân cụm (Clustering Algorithms)	5
1.1 Phương pháp hình thành cụm	5
1.2 Các thuật toán phân cụm chính	6
1.3 Đo hiệu năng phân cụm	6
1.3.1 Phân tích Silhouette (Silhouette Analysis)	6
1.3.2 Davies–Bouldin Index	7
1.3.3 Dunn Index	7
1.4 Ứng dụng phân cụm	7
2 Phân cụm dựa trên centroid (Centroid-Based Clustering)	8
2.1 K-means Clustering	8
2.2 K-Medoids Clustering	8
3 Thuật toán K-Means	9
3.1 Cách hoạt động của K-Means	9
3.2 Mục tiêu phân tích cụm	10
3.3 Triển khai K-Means bằng Python	10
3.3.1 Ví dụ 1 — Hiểu K-Means với dữ liệu ngẫu nhiên	10
3.3.2 Ví dụ 2 — Dataset 2D bốn blob	12
3.3.3 Ví dụ 3 — Chữ số viết tay (digits)	14
3.4 Ưu điểm K-Means	16
3.5 Nhược điểm K-Means	16
3.6 Ứng dụng K-Means	16
4 Phân cụm K-Medoids	16
4.1 Thuật toán K-medoids	17
4.2 Triển khai bằng Python	17
4.3 Ưu điểm K-medoids	18
4.4 Nhược điểm K-medoids	19
5 Thuật toán Mean-Shift	19
5.1 Cách hoạt động Mean-Shift	19
5.2 Triển khai Mean-Shift trong Python	20
5.2.1 Ví dụ 2 — Dataset 2D nhiều blob	21
5.3 Ứng dụng Mean-Shift	22
5.4 Ưu điểm và nhược điểm	22
6 Phân cụm phân cấp (Hierarchical Clustering)	23
6.1 Các bước phân cụm phân cấp gộp (Agglomerative)	23
6.2 Vai trò dendrogram	23
6.2.1 Ví dụ — Dendrogram	23
6.2.2 Ví dụ — Cụm trên dữ liệu y khoa	27

7 Phân cụm dựa trên mật độ (Density-Based Clustering)	29
7.1 Các thuật toán phân cụm theo mật độ phổ biến	29
7.1.1 DBSCAN	29
7.1.2 OPTICS	29
7.1.3 HDBSCAN	30
8 DBSCAN Clustering	30
8.1 Cách DBSCAN hoạt động	30
8.2 Triển khai Python (scikit-learn)	31
8.3 Ưu và nhược điểm	32
9 OPTICS Clustering	32
9.1 Triển khai Python (scikit-learn)	33
9.2 Ưu và nhược điểm	34
10 HDBSCAN Clustering	34
10.1 Cách HDBSCAN hoạt động	34
10.2 Cài đặt và sử dụng (gói hdbscan)	35
10.3 Tham số chính	35
11 BIRCH Clustering	36
11.1 Ba bước chính	36
11.2 Triển khai Python (scikit-learn)	36
11.3 Ưu và nhược điểm	37
12 Affinity Propagation	38
12.1 Responsibility và Availability	38
12.2 Triển khai Python (scikit-learn)	38
12.3 Ưu và nhược điểm	39
13 Phân cụm dựa trên phân phối (Distribution-Based)	39
13.1 Gaussian Mixture Model (GMM)	40
13.2 Triển khai Python (GaussianMixture, covariance full)	40
13.3 Ưu và nhược điểm GMM	41
14 Agglomerative Clustering	41
14.1 Các bước thuật toán	41
14.2 Triển khai Python (Iris, dendrogram)	42
14.3 Ưu, nhược điểm và ứng dụng	43

Lời nói đầu

Nguồn

Tài liệu biên soạn và dịch đầy đủ từ Tutorialspoint Machine Learning — mục Clustering Algorithms: https://www.tutorialspoint.com/machine_learning/machine_learning_clustering_algorithms.htm.

Cách dùng

- Đọc sau phần Classification Algorithms.

Dặng Tấn Quý

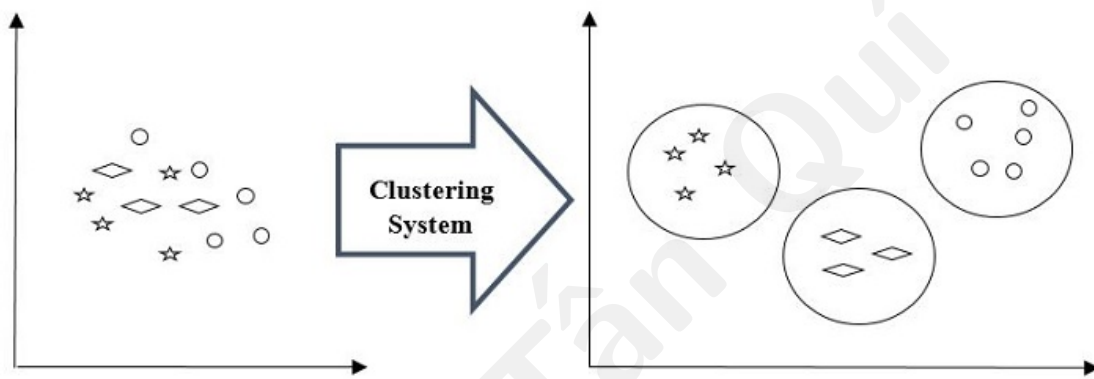
1 Thuật toán phân cụm (Clustering Algorithms)

Thuật toán phân cụm

Là một trong những phương pháp **học không giám sát** hữu ích nhất: tìm độ tương tự và quy luật quan hệ giữa các mẫu dữ liệu, rồi gom các mẫu tương tự (theo feature) thành cụm.

Tầm quan trọng

Phân cụm xác định nhóm nội tại trong dữ liệu **không nhãn**. Mỗi giả định về độ tương tự giữa điểm dữ liệu tạo ra các cụm khác nhau nhưng đều có thể hợp lệ.



1.1 Phương pháp hình thành cụm

Không nhất thiết hình cầu

Cụm không bắt buộc có dạng hình cầu. Các hướng hình thành cụm phổ biến:

- **Dựa trên mật độ (Density-based)**: Cụm là vùng **mật độ cao**; ưu điểm là độ chính xác cao, có thể gộp hai cụm.
- **Phân cấp (Hierarchical-based)**: Cụm dạng **cây** theo thứ bậc. Cụm kết hợp - Agglomerative (gộp từ dưới lên), cụm phân chia - Divisive (chia từ trên xuống).
- **Phân hoạch (Partitioning)**: Chia đối tượng thành k cụm; số cụm bằng số phân hoạch.
- **Lưới (Grid)**: Cụm dạng **lưới**; ưu điểm là thao tác trên lưới nhanh, độc lập số đối tượng.

1.2 Các thuật toán phân cụm chính

Danh sách

- **K-Means**: Tính centroid và lặp đến centroid tối ưu; giả định **đã biết** số cụm K (flat clustering).
- **K-Medoids**: Cải tiến K-means: chọn k medoid ngẫu nhiên, gán điểm, cập nhật medoid (tối thiểu tổng khoảng cách trong cụm), lặp đến hội tụ.
- **Mean-Shift**: Phân cụm mạnh, **không tham số hóa** giả định phân phối (non-parametric).
- **DBSCAN**: Dựa trên mật độ phổ biến nhất. Cần hai tham số: **minPts** (số láng giềng tối thiểu) và **eps** (khoảng cách tối đa giữa điểm lõi).
- **OPTICS**: Giống DBSCAN nhưng xử lý cụm mật độ khác nhau, nhiều, và cho cấu trúc phân cấp.
- **HDBSCAN**: Mở rộng DBSCAN: cụm mật độ thay đổi, hình dạng/kích thước đa dạng.
- **BIRCH**: Phân cấp cho dataset lớn; cây cụm con bằng chia đệ quy đến khi đạt tiêu chí dừng.
- **Lan truyền liên kết (Affinity Propagation)**: Xác định “mẫu đại diện” (exemplar); không cần chỉ định trước số cụm — hữu ích khám phá dữ liệu.
- **Phân cấp kết hợp (Agglomerative Clustering)**: Phân cấp từ dưới lên: mỗi điểm là một cụm, gộp cụm gần nhất; sinh **cây phân cấp** (dendrogram).
- **Mô hình hỗn hợp Gaussian (Gaussian Mixture Model)**: Giả định dữ liệu sinh từ hỗn hợp phân phối Gaussian; mỗi Gaussian là một cụm.

1.3 Đo hiệu năng phân cụm

Bối cảnh

Với học có giám sát, đánh giá mô hình tương đối dễ vì có nhãn. Với học không giám sát, dữ liệu không nhãn nhưng vẫn có metric so sánh mô hình với nhau (không chứng minh “đúng tuyệt đối”).

1.3.1 Phân tích Silhouette (Silhouette Analysis)

Silhouette score

Đo khoảng cách giữa các cụm; hỗ trợ chọn số cụm qua điểm Silhouette. Miền giá trị $[-1, 1]$:

- Gần $+1$: mẫu xa cụm láng giềng.
- 0 : mẫu gần ranh giới giữa hai cụm.
- Gần -1 : mẫu có thể gán nhầm cụm.

Công thức

Với mỗi mẫu: $s = \frac{b-a}{\max(a,b)}$, trong đó a = khoảng cách trung bình trong cụm, b = khoảng cách trung bình đến cụm láng giềng gần nhất.

1.3.2 Davies–Bouldin Index**DB index**

Đánh giá cụm có **cách xa** nhau không, cụm có **dày** không; DB index **càng nhỏ** càng tốt.

$$DB = \frac{1}{n} \sum_{i=1}^n \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right)$$

n : số cụm; σ_i : khoảng cách trung bình các điểm trong cụm đến centroid c_i .

1.3.3 Dunn Index**Dunn Index**

Tương tự DB index nhưng:

- Dunn chỉ xét **trường hợp xấu nhất** (cụm gần nhau nhất).
- DB xét **độ phân tán** (dispersion) và **độ tách biệt** (separation) của **mọi** cặp cụm.
- Dunn **tăng** khi hiệu năng tốt; DB tốt khi cụm tách xa và dày.

1.4 Ứng dụng phân cụm**Ứng dụng phân cụm**

- Tóm tắt và nén dữ liệu: xử lý ảnh, lượng tử hóa vector.
- Gom sản phẩm hoặc người dùng tương tự: hệ thống gợi ý và phân khúc khách hàng.
- Bước trung gian khai phá dữ liệu: tóm tắt cho phân loại, kiểm định, sinh giả thuyết.
- Gom cụm xu hướng tương tự theo thời gian: phát hiện xu hướng trong dữ liệu động.
- Phân tích mạng xã hội và dữ liệu sinh học: chuỗi trong ảnh/video/âm thanh; phân cụm trong phân tích dữ liệu sinh học.

2 Phân cụm dựa trên centroid (Centroid-Based Clustering)

Phân cụm dựa trên centroid

Phân cụm dựa trên centroid là lớp thuật toán học máy nhằm **phân hoạch** tập dữ liệu thành các nhóm (cụm) dựa trên **khoảng cách** từ điểm dữ liệu đến centroid của từng cụm.

Centroid là gì?

Centroid của một cụm là **trung bình cộng số học** của mọi điểm trong cụm đó; nó đóng vai trò điểm đại diện cho cụm.

Hai thuật toán phổ biến nhất

Hai thuật toán phân cụm dựa trên centroid được dùng rộng rãi nhất là **K-means** và **K-medoids**.

2.1 K-means Clustering

K-means

K-means là thuật toán **học không giám sát** phổ biến để phân cụm dữ liệu: đơn giản, hiệu quả, gom điểm vào K cụm theo độ tương tự.

Cơ chế tóm tắt

1. Chọn ngẫu nhiên K centroid ban đầu — tâm khởi tạo của mỗi cụm.
2. Gán mỗi điểm vào cụm có centroid **gần nhất**.
3. Cập nhật centroid bằng **trung bình** các điểm trong cụm.
4. Lặp đến khi centroid không đổi hoặc đạt số vòng lặp tối đa.

2.2 K-Medoids Clustering

K-medoids

K-medoids là thuật toán phân hoạch gom tập điểm thành k cụm. Khác K-means dùng **giá trị trung bình** làm tâm cụm, K-medoids dùng một **điểm dữ liệu thực** gọi là **medoid** làm đại diện.

Medoid

Medoid là điểm trong cụm **tối thiểu hóa tổng khoảng cách** tới mọi điểm khác trong cụm đó.

So với K-means

K-medoids **bền hơn** với ngoại lai và nhiễu vì tâm cụm luôn là một mẫu quan sát được, không phải trung bình có thể bị kéo lệch bởi điểm cực đoan.

3 Thuật toán K-Means

K-Means Clustering

Thuật toán K-Means tính centroid và lặp cho đến khi đạt centroid tối ưu. Giả định **đã biết trước** số cụm; còn gọi là thuật toán phân cụm **phẳng** (flat clustering). Ký hiệu K là số cụm mà thuật toán xác định từ dữ liệu.

Mục tiêu tối thiểu

Mỗi điểm được gán vào cụm sao cho **tổng bình phương khoảng cách** giữa điểm và centroid là nhỏ nhất. Biến thiên trong cụm càng nhỏ thì các điểm trong cùng cụm càng giống nhau.

3.1 Cách hoạt động của K-Means

Các bước

- **Bước 1:** Chỉ định số cụm K cần sinh.
- **Bước 2:** Chọn ngẫu nhiên K điểm và gán ban đầu (phân loại theo số điểm khởi tạo).
- **Bước 3:** Tính centroid của từng cụm.
- **Bước 4:** Lặp đến khi phân gán ổn định:
 - **4.1:** Tính tổng bình phương khoảng cách giữa điểm và centroid.
 - **4.2:** Gán mỗi điểm vào cụm có centroid **gần hơn** các cụm còn lại.
 - **4.3:** Cập nhật centroid bằng **trung bình** mọi điểm trong cụm.

Expectation–Maximization (EM)

K-means theo khung EM:

- **Bước E (Expectation - Kỳ vọng):** gán điểm vào cụm gần nhất.
- **Bước M (Maximization - Cực đại hóa):** tính lại centroid mỗi cụm.

Chuẩn hóa dữ liệu

Thuật toán dùng đo **khoảng cách** để đo tương tự; nên **chuẩn hóa** (standardize) feature trước khi chạy K-means và các thuật toán phân cụm tương tự.

Cục cực địa phương

Do khởi tạo centroid ngẫu nhiên và lặp lặp lại, K-means có thể kẹt ở **cục cực địa phương** thay vì toàn cục. Nên thử **nhiều khởi tạo** centroid khác nhau (ví dụ `n_init` trong `scikit-learn`).

Ưu/nhược điểm tổng quan

Thuật toán đơn giản, hiệu quả với dataset lớn; nhưng nhạy khởi tạo, dễ hội tụ cục bộ, và thường giả định phương sai đồng đều giữa các cụm.

3.2 Mục tiêu phân tích cụm

Hai mục tiêu chính

- Rút ra **trực giác** có ý nghĩa từ dữ liệu.
- **Phân cụm rồi dự đoán**: xây mô hình riêng cho từng nhóm con sau khi phân cụm.

3.3 Triển khai K-Means bằng Python

scikit-learn

Python có `scikit-learn` với lớp `KMeans` trong `sklearn.cluster`.

3.3.1 Ví dụ 1 — Hiểu K-Means với dữ liệu ngẫu nhiên

Bối cảnh

Sinh 300 điểm hai chiều ngẫu nhiên, áp dụng K-means với $K = 3$.

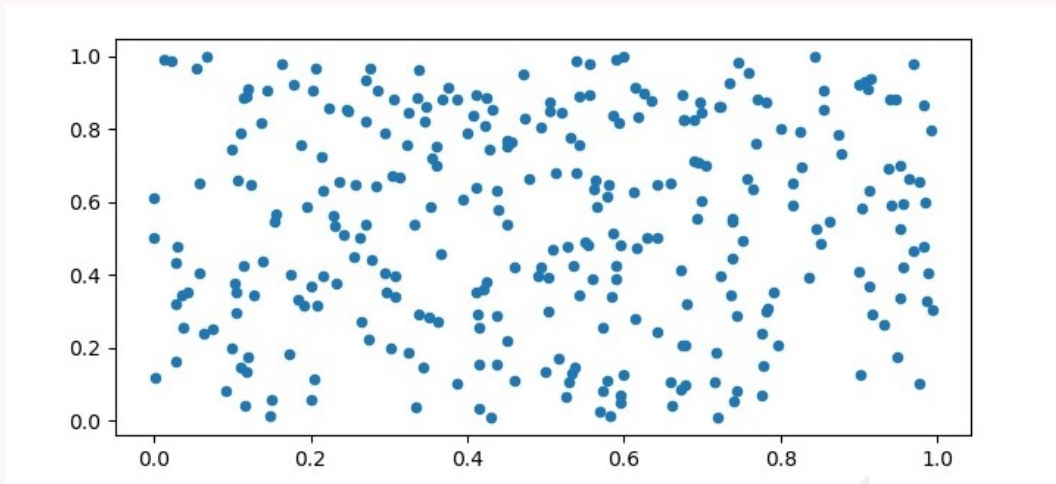
Bước 1 — Import

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

Bước 2 — Sinh và vẽ dữ liệu

```
X = np.random.rand(300, 2)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], s=20)
plt.show()
```

Dữ liệu ban đầu**Bước 3 — Khởi tạo K-Means**

```
kmeans = KMeans(n_clusters=3, max_iter=100)
```

Bước 4 — Huấn luyện

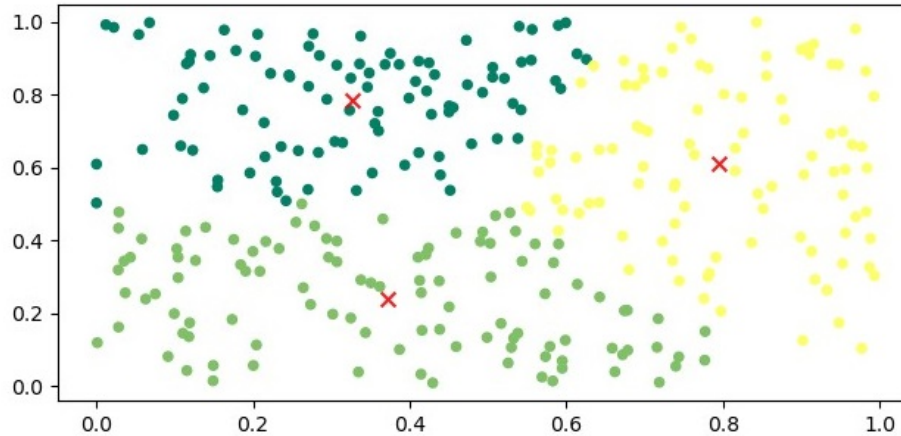
```
kmeans.fit(X)
```

Bước 5 — Vẽ cụm và centroid

```
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, s=20)
plt.scatter(kmeans.cluster_centers_[0, 0],
            kmeans.cluster_centers_[0, 1],
            marker='x', c='r', s=50, alpha=0.9)
plt.show()
```

Kết quả phân cụm

Điểm tô màu theo nhãn cụm; centroid đánh dấu x màu đỏ.



Code đầy đủ (ví dụ 1)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

X = np.random.rand(300, 2)

kmeans = KMeans(n_clusters=3, max_iter=100)
kmeans.fit(X)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_, s=20)
plt.scatter(kmeans.cluster_centers_[0, 0],
            kmeans.cluster_centers_[0, 1],
            marker='x', c='r', s=50, alpha=0.9)
plt.show()
```

3.3.2 Ví dụ 2 — Dataset 2D bốn blob

Bối cảnh

Sinh dataset 2D gồm 4 blob bằng `make_blobs`, huấn luyện K-means với 4 cụm.

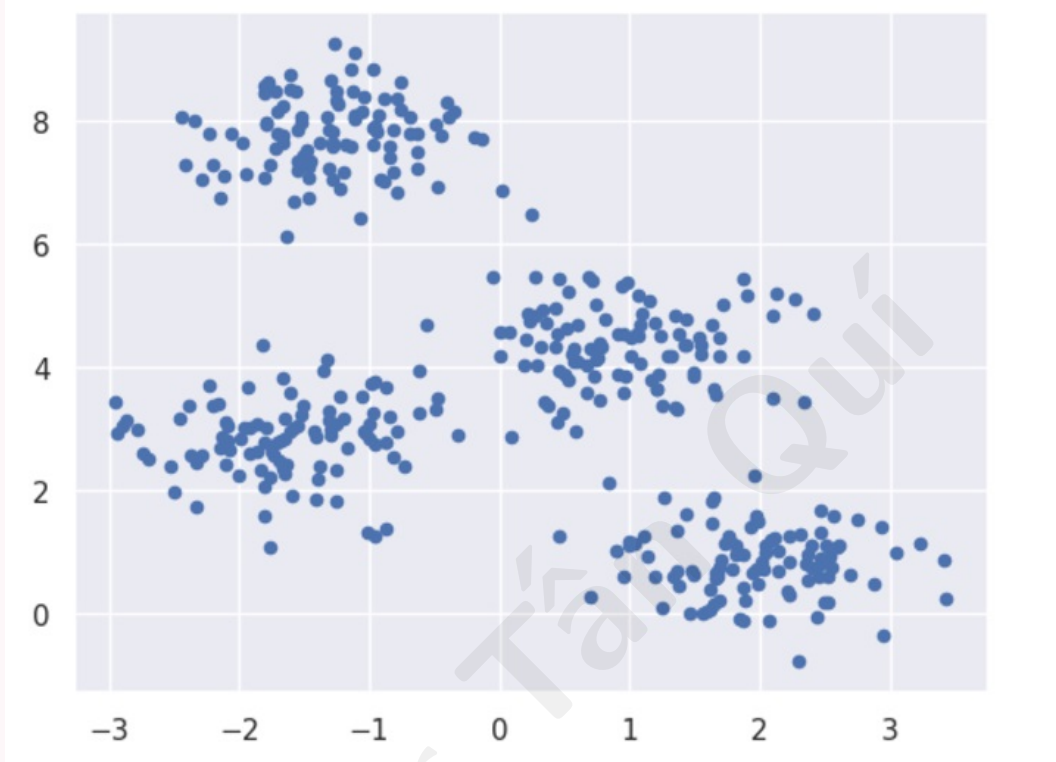
Import và sinh blob

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, y_true = make_blobs(n_samples=400, centers=4,
                       cluster_std=0.60, random_state=0)
```

Vẽ dữ liệu gốc

```
plt.scatter(X[:, 0], X[:, 1], s=20)
plt.show()
```

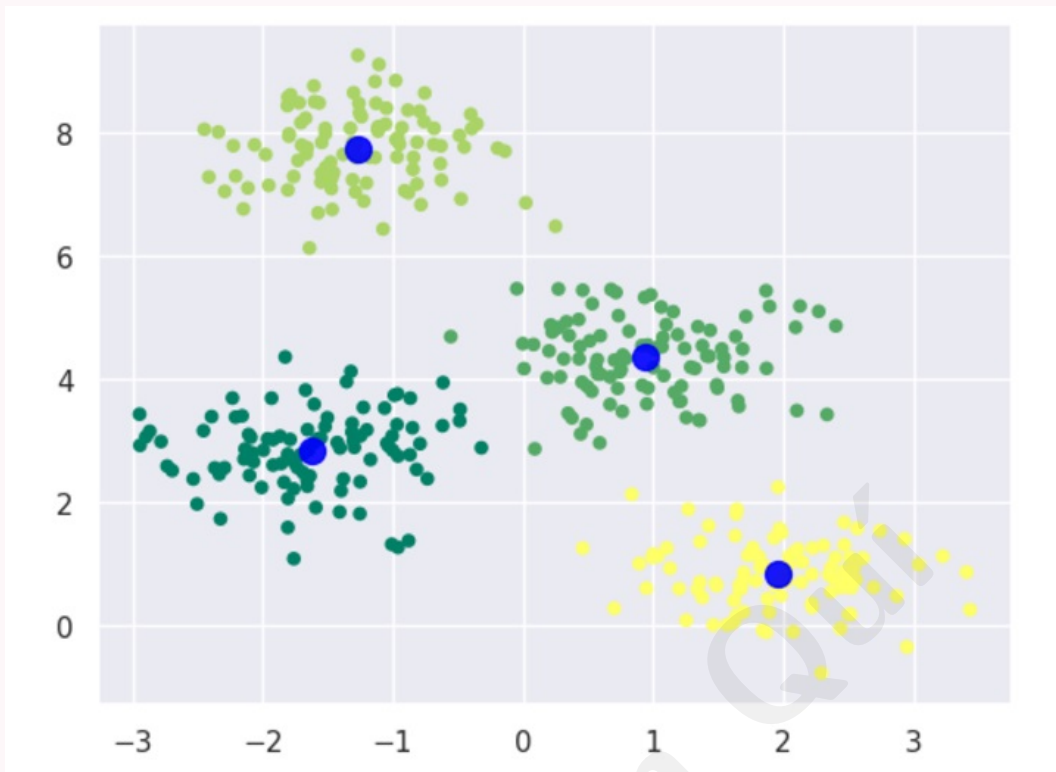
Trực quan 2D blob**Huấn luyện và dự đoán**

```
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
```

Vẽ cụm và tâm

```
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=20)
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1],
            c='blue', s=100, alpha=0.9)
plt.show()
```

Centroid học được



3.3.3 Ví dụ 3 — Chữ số viết tay (digits)

Bối cảnh

Áp dụng K-means lên `load_digits`: tìm nhóm chữ số tương tự **không** dùng nhãn gốc khi gom cụm.

Nạp dữ liệu

```
from sklearn.datasets import load_digits

digits = load_digits()
digits.data.shape  # (1797, 64)
```

Kích thước

1797 mẫu, mỗi mẫu 64 feature (ảnh 8×8).

Phân cụm 10 nhóm

```
kmeans = KMeans(n_clusters=10, random_state=0)
clusters = kmeans.fit_predict(digits.data)
kmeans.cluster_centers_.shape  # (10, 64)
```

Vẽ tâm cụm dạng ảnh

```
fig, ax = plt.subplots(2, 5, figsize=(8, 3))
centers = kmeans.cluster_centers_.reshape(10, 8, 8)
for axi, center in zip(ax.flat, centers):
    axi.set(xticks=[], yticks=[])
    axi.imshow(center, interpolation='nearest',
               cmap=plt.cm.binary)
```

Centroid mỗi cụm



Ghép nhãn cụm với nhãn thật

Dùng `scipy.stats.mode` để gán mỗi cụm nhãn digit phổ biến nhất trong cụm đó, rồi tính `accuracy_score`.

Gán nhãn và độ chính xác

```
from scipy.stats import mode
from sklearn.metrics import accuracy_score

labels = np.zeros_like(clusters)
for i in range(10):
    mask = (clusters == i)
    labels[mask] = mode(digits.target[mask], keepdims=True).mode[0]

accuracy_score(digits.target, labels) # ~0.79
```

Độ chính xác 80%

Ghép nhãn cụm với nhãn gốc cho độ chính xác khoảng **80%** — minh họa cụm bắt được cấu trúc chữ số dù không dùng nhãn lúc fit.

3.4 Ưu điểm K-Means

Danh sách ưu điểm

- Dễ hiểu và triển khai.
- Với số biến lớn, thường **nhANH hơn** phân cụm phân cấp.
- Khi tính lại centroid, một mẫu có thể **đổi cụm**.
- Cụm thường **chẶT** hơn so với phân cụm phân cấp trong nhiều bài toán.

3.5 Nhược điểm K-Means

Hạn chế

- Khó chọn k (số cụm).
- Kết quả phụ thuộc mạnh vào k và khởi tạo.
- Thứ tự dữ liệu có thể ảnh hưởng kết quả cuối (với một số triển khai).
- **Nhạy với chuẩn hóa (rescale)**: chuẩn hóa/chuẩn tắc hóa làm thay đổi hoàn toàn phân cụm — cần xử lý nhất quán.
- Kém khi cụm có **hình học phức tạp** (không lồi, không cầu).

3.6 Ứng dụng K-Means

Ứng dụng K-Means

- Phân đoạn ảnh: Gom vùng pixel theo màu hoặc texture; dùng trong nhận dạng đối tượng, truy vấn ảnh, ảnh y khoa.
- Phân khúc khách hàng: Chia khách theo hành vi mua hoặc nhân khẩu; hỗ trợ giữ chân khách, loyalty, quảng cáo nhắm mục tiêu.
- Phát hiện bất thường: Điểm không thuộc cụm nào (hoặc xa mọi centroid) có thể là bất thường (anomaly) — gian lận, xâm nhập mạng, bảo trì dự đoán.
- Phân tích biểu hiện gen: Gom gen đồng điều hòa/biểu hiện; hỗ trợ khám phá thuốc, chẩn đoán, y học cá thể hóa.

4 Phân cụm K-Medoids

K-medoids

K-medoids (PAM — Partitioning Around Medoids) là thuật toán phân hoạch: chọn k điểm thực làm tâm cụm, gán điểm, cập nhật medoid, lặp đến hội tụ.

4.1 Thuật toán K-medoids

Các bước

1. **Khởi tạo k medoid**: chọn ngẫu nhiên k điểm dữ liệu làm medoid ban đầu.
2. **Gán điểm**: mỗi điểm được gán vào medoid **gần nhất**.
3. **Cập nhật medoid**: với mỗi cụm, chọn điểm **tối thiểu tổng khoảng cách** tới mọi điểm khác trong cụm; đặt làm medoid mới.
4. **Lặp** bước 2 và 3 đến khi hội tụ hoặc đạt số vòng lặp tối đa.

4.2 Triển khai bằng Python

Thư viện

Triển khai K-medoids trong Python thường dùng gói `sklearn-extra`: lớp `KMedoids` trong `sklearn_extra.cluster`.

Import và tạo dữ liệu mẫu

```
from sklearn_extra.cluster import KMedoids
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, y = make_blobs(n_samples=500, centers=3, random_state=42)
```

Dataset `make_blobs`

Ví dụ trên sinh 500 điểm, 3 cụm, `random_state=42` để tái lập kết quả.

Huấn luyện `KMedoids`

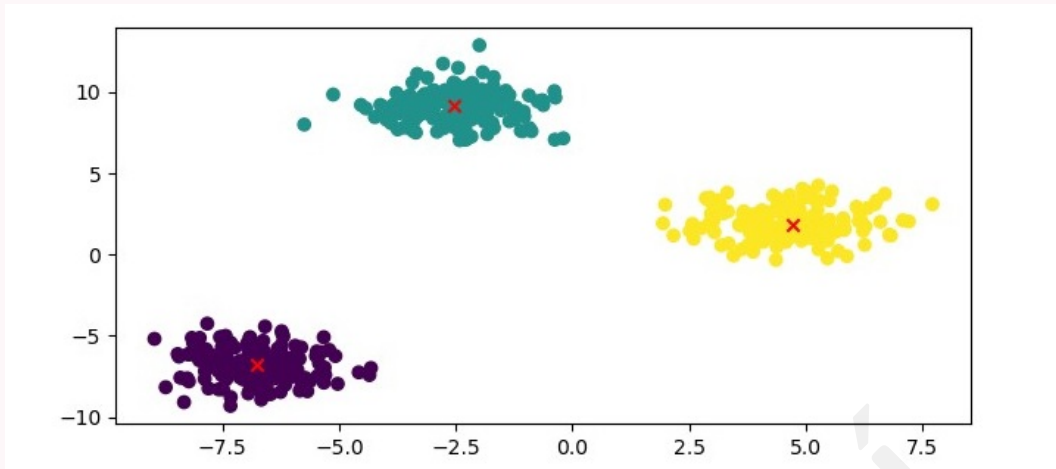
```
kmedoids = KMedoids(n_clusters=3, random_state=42)
kmedoids.fit(X)
```

Vẽ kết quả phân cụm

```
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=kmedoids.labels_, cmap='viridis')
plt.scatter(kmedoids.cluster_centers_[0, 0],
            kmedoids.cluster_centers_[0, 1],
            marker='x', color='red')
plt.show()
```

Đồ thị kết quả

Điểm được tô màu theo nhãn cụm; medoid hiển thị bằng dấu x màu đỏ.

**Code đầy đủ**

```
from sklearn_extra.cluster import KMedoids
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, y = make_blobs(n_samples=500, centers=3, random_state=42)

kmedoids = KMedoids(n_clusters=3, random_state=42)
kmedoids.fit(X)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=kmedoids.labels_, cmap='viridis')
plt.scatter(kmedoids.cluster_centers_[0, 0],
            kmedoids.cluster_centers_[0, 1],
            marker='x', color='red')
plt.show()
```

4.3 Ưu điểm K-medoids**Ưu điểm K-medoids**

- Bền với ngoại lai và nhiễu: Dùng medoid (điểm thực) thay vì trung bình $\Rightarrow$ ít bị kéo bởi outlier hơn K-means.
- Metric khoảng cách linh hoạt: Có thể dùng metric không phải Euclidean: Manhattan, cosine similarity, v.v.
- Độ phức tạp: Độ phức tạp tính toán $O(k \cdot n^2)$; trong một số bối cảnh so sánh được với chi phí K-means khi n vừa phải.

4.4 Nhược điểm K-medoids

Nhược điểm K-medoids

- Nhạy với k : Hiệu năng phụ thuộc mạnh vào lựa chọn k (số cụm).
- Dữ liệu chiều cao: Trên dữ liệu **nhiều chiều**, bước chọn medoid tốn kém và chất lượng cụm có thể kém.

5 Thuật toán Mean-Shift

Mean-Shift Clustering

Mean-Shift là thuật toán phân cụm **không tham số hóa** (non-parametric): lặp dịch chuyển trung bình của điểm về vùng **mật độ cao nhất** của dữ liệu. Mật độ được định nghĩa qua **hàm nhân** (kernel), thường là Gaussian, gán trọng số theo khoảng cách tối mean hiện tại.

Không cần chỉ định K trước

Khác K-means phải cho biết số cụm, Mean-Shift **tự suy ra** số cụm từ dữ liệu và tham số bandwidth.

Phân cụm theo mật độ

Thuật toán thuộc nhóm density-based: cụm là vùng mật độ điểm cao, không chỉ dựa vào khoảng cách Euclidean đơn thuần giữa hai điểm xa nhau.

5.1 Cách hoạt động Mean-Shift

Các bước

- **Bước 1:** Ban đầu mỗi điểm (hoặc mỗi seed) được coi là một cụm riêng.
- **Bước 2:** Tính centroid (mean) có trọng số kernel quanh từng điểm.
- **Bước 3:** Cập nhật vị trí centroid mới.
- **Bước 4:** Lặp, dịch chuyển về vùng mật độ cao hơn.
- **Bước 5:** Dừng khi centroid không còn dịch chuyển đáng kể.

5.2 Triển khai Mean-Shift trong Python

scikit-learn

Dùng `MeanShift` và `estimate_bandwidth` trong `sklearn.cluster`.

Import

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MeanShift, estimate_bandwidth
```

Sinh dữ liệu và ước lượng bandwidth

```
X = np.random.randn(500, 2)

bandwidth = estimate_bandwidth(X, quantile=0.1, n_samples=100)
```

Bandwidth

Bandwidth quyết định **độ rộng** kernel; ảnh hưởng trực tiếp số cụm và độ mịn của ranh giới.

Khởi tạo, huấn luyện, trực quan

```
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(X)

labels = ms.labels_
cluster_centers = ms.cluster_centers_
n_clusters_ = len(np.unique(labels))
print("Number of estimated clusters:", n_clusters_)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(cluster_centers[:, 0], cluster_centers[:, 1],
            marker='*', s=300, c='r')
plt.show()
```

Ví dụ đầy đủ (dữ liệu Gaussian 2D)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MeanShift, estimate_bandwidth

X = np.random.randn(500, 2)
bandwidth = estimate_bandwidth(X, quantile=0.1, n_samples=100)
ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(X)

labels = ms.labels_
cluster_centers = ms.cluster_centers_
print("Number of estimated clusters:", len(np.unique(labels)))
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='summer')
plt.scatter(cluster_centers[:, 0], cluster_centers[:, 1],
            marker='*', s=200, c='r')
plt.show()
```

5.2.1 Ví dụ 2 — Dataset 2D nhiều blob

make_blobs 3D trong mã nguồn

Một số ví dụ dùng make_blobs với tâm 3 chiều; có thể chiếu hai cột đầu để vẽ 2D hoặc dùng centers=4 hai chiều tương đương bài minh họa 4 cụm.

Sinh blob và vẽ

```
from sklearn.datasets import make_blobs

X, _ = make_blobs(n_samples=700, centers=4,
                  cluster_std=0.5, random_state=0)
plt.scatter(X[:, 0], X[:, 1])
plt.show()
```

Mean-Shift mặc định và in tâm

```
ms = MeanShift()
ms.fit(X)
labels = ms.labels_
cluster_centers = ms.cluster_centers_
print(cluster_centers)
print("Estimated clusters:", len(np.unique(labels)))
```

Vẽ từng điểm theo nhãn

```
colors = 10 * ['r.', 'g.', 'b.', 'c.', 'k.', 'y.', 'm.']
for i in range(len(X)):
    plt.plot(X[i][0], X[i][1], colors[labels[i]], markersize=3)
plt.scatter(cluster_centers[:, 0], cluster_centers[:, 1],
            marker='.', color='k', s=20, linewidths=5, zorder=10)
plt.show()
```

Kết quả mẫu

Ví dụ trên có thể ước lượng khoảng **3 cụm** với ba tâm cụm in ra từ cluster_centers_.

5.3 Ứng dụng Mean-Shift

Ứng dụng Mean-Shift

- Thị giác máy tính: Theo dõi đối tượng, phân đoạn ảnh, trích xuất đặc trưng.
- Xử lý ảnh: Phân đoạn ảnh theo độ tương đồng pixel.
- Phát hiện bất thường: Vùng mật độ thấp có thể đánh dấu bất thường.
- Phân khúc khách hàng và mạng xã hội: Gom nhóm hành vi hoặc sở thích người dùng tương tự.

5.4 Ưu điểm và nhược điểm

Ưu điểm

- Không giả định mô hình phân phối cố định như K-means hay GMM.
- Mô hình được cụm **lồi và phức tạp** (non-convex).
- Chỉ cần điều chỉnh chủ yếu tham số **bandwidth**; số cụm suy ra từ dữ liệu.
- Ít vấn đề **cực cực địa phương** như K-means.
- Tương đối ít bị ngoại lai kéo mean toàn cục như trung bình K-means.

Nhược điểm

- Chi phí tính toán cao trên dữ liệu **nhiều chiều**; số cụm có thể thay đổi đột ngột khi bandwidth không phù hợp.
- Không kiểm soát trực tiếp số cụm — một số bài toán cần K cố định.
- Khó phân biệt mode “có ý nghĩa” và mode nhiễu trên bề mặt mật độ.

6 Phân cụm phân cấp (Hierarchical Clustering)

Phân cụm phân cấp

Phân cụm phân cấp là thuật toán học **không giám sát** gom các điểm **không nhãn** có đặc tính tương tự. Cấu trúc cụm được biểu diễn dạng **cây** hoặc **dendrogram**.

Hai họ thuật toán

- **Agglomerative** (gộp từ dưới lên): mỗi điểm ban đầu là một cụm, lần lượt gộp cặp cụm gần nhất.
- **Divisive** (chia từ trên xuống): ban đầu một cụm lớn, lần lượt chia thành cụm nhỏ hơn.

6.1 Các bước phân cụm phân cấp gộp (Agglomerative)

Quy trình

- **Bước 1:** Coi mỗi điểm là một cụm — ban đầu có K cụm bằng số điểm.
- **Bước 2:** Gộp hai điểm **gần nhất** thành một cụm $\Rightarrow$ còn $K - 1$ cụm.
- **Bước 3:** Tiếp tục gộp hai cụm gần nhất $\Rightarrow K - 2$ cụm.
- **Bước 4:** Lặp đến khi còn một cụm duy nhất (mọi điểm đã được gộp).
- **Bước 5:** Dùng **dendrogram** để **cắt** cây thành số cụm phù hợp bài toán.

6.2 Vai trò dendrogram

Dendrogram sau khi gộp hết

Khi đã có một cụm lớn duy nhất, dendrogram cho phép **tách** lại thành nhiều cụm con có quan hệ gần gũi, tùy ngưỡng cắt hoặc số cụm mong muốn.

6.2.1 Ví dụ — Dendrogram

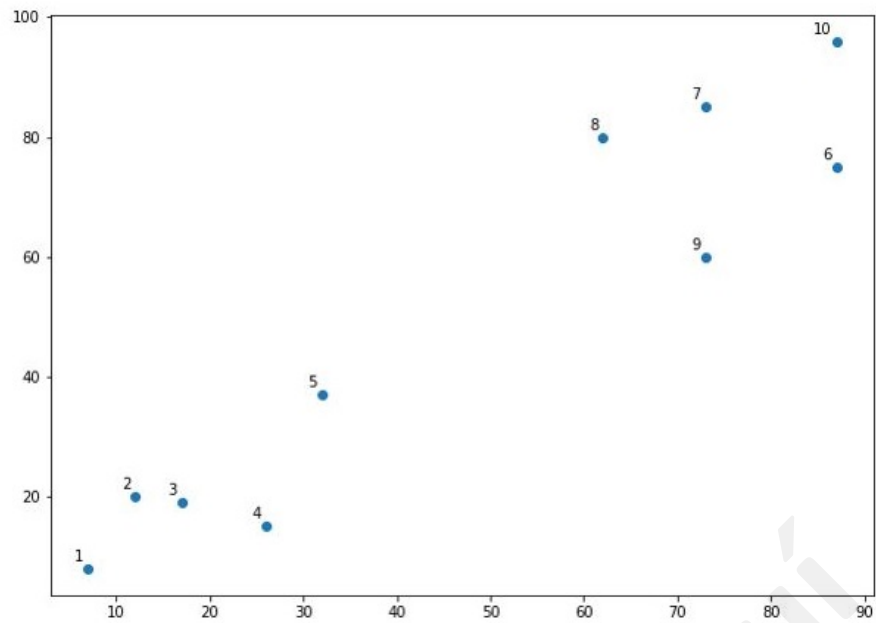
Import và 10 điểm 2D

```
import matplotlib.pyplot as plt
import numpy as np

X = np.array([[7, 8], [12, 20], [17, 19], [26, 15], [32, 37],
              [87, 75], [73, 85], [62, 80], [73, 60], [87, 96]])
labels = range(1, 11)
plt.figure(figsize=(10, 7))
plt.subplots_adjust(bottom=0.1)
plt.scatter(X[:, 0], X[:, 1], label='True Position')
for label, x, y in zip(labels, X[:, 0], X[:, 1]):
    plt.annotate(label, xy=(x, y), xytext=(-3, 3),
                 textcoords='offset points', ha='right', va='bottom')
plt.show()
```

Hai nhóm trực quan

Trên đồ thị dễ thấy hai nhóm điểm; dữ liệu thực tế có thể có hàng nghìn cụm tiềm ẩn.

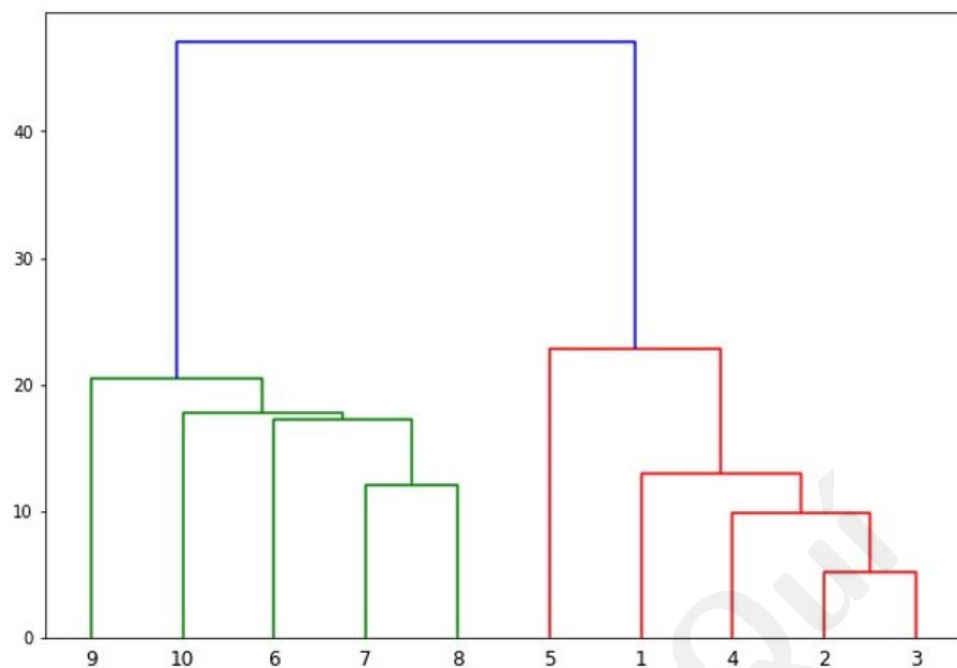


Vẽ dendrogram với SciPy

```
from scipy.cluster.hierarchy import dendrogram, linkage

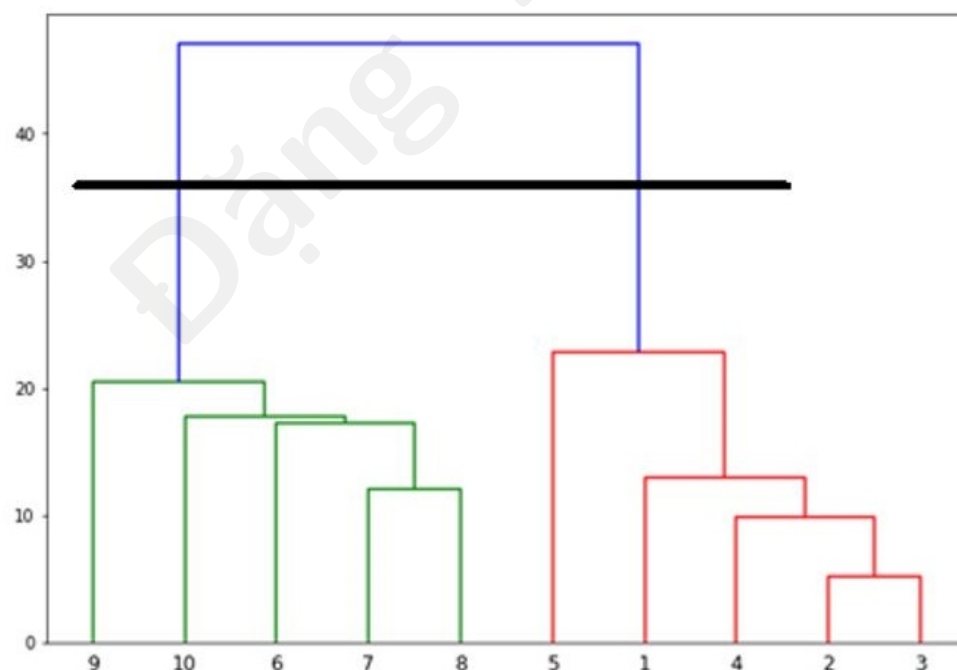
linked = linkage(X, 'single')
labellist = range(1, 11)
plt.figure(figsize=(10, 7))
dendrogram(linked, orientation='top', labels=labellist,
            distance_sort='descending', show_leaf_counts=True)
plt.show()
```

Dendrogram linkage single



Chọn số cụm bằng khoảng cách dọc lớn nhất

Chọn **đoạn dọc dài nhất** trên dendrogram, kẻ đường ngang cắt qua: số lần cắt cho biết số cụm. Ví dụ đường ngang cắt hai nhánh $\Rightarrow$ **2 cụm**.



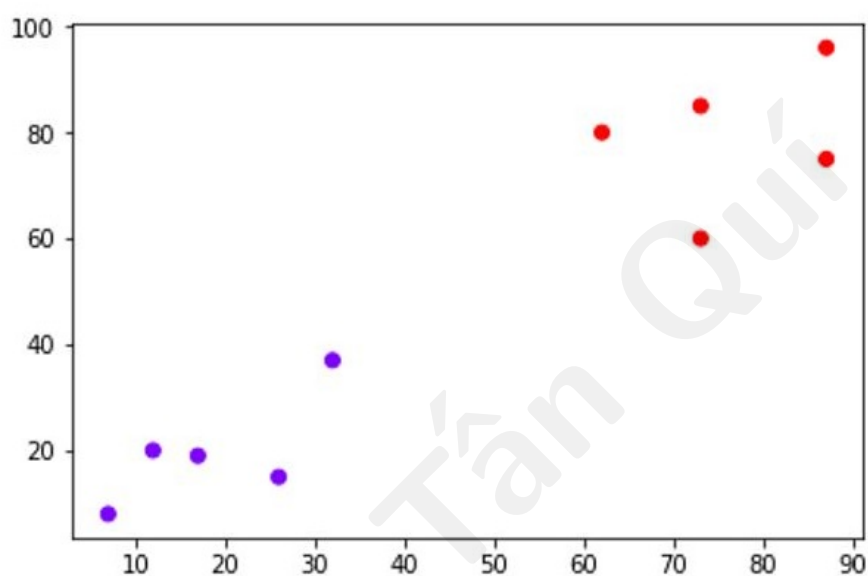
AgglomerativeClustering (sklearn)

```
from sklearn.cluster import AgglomerativeClustering

cluster = AgglomerativeClustering(n_clusters=2, linkage='ward')
cluster.fit_predict(X)

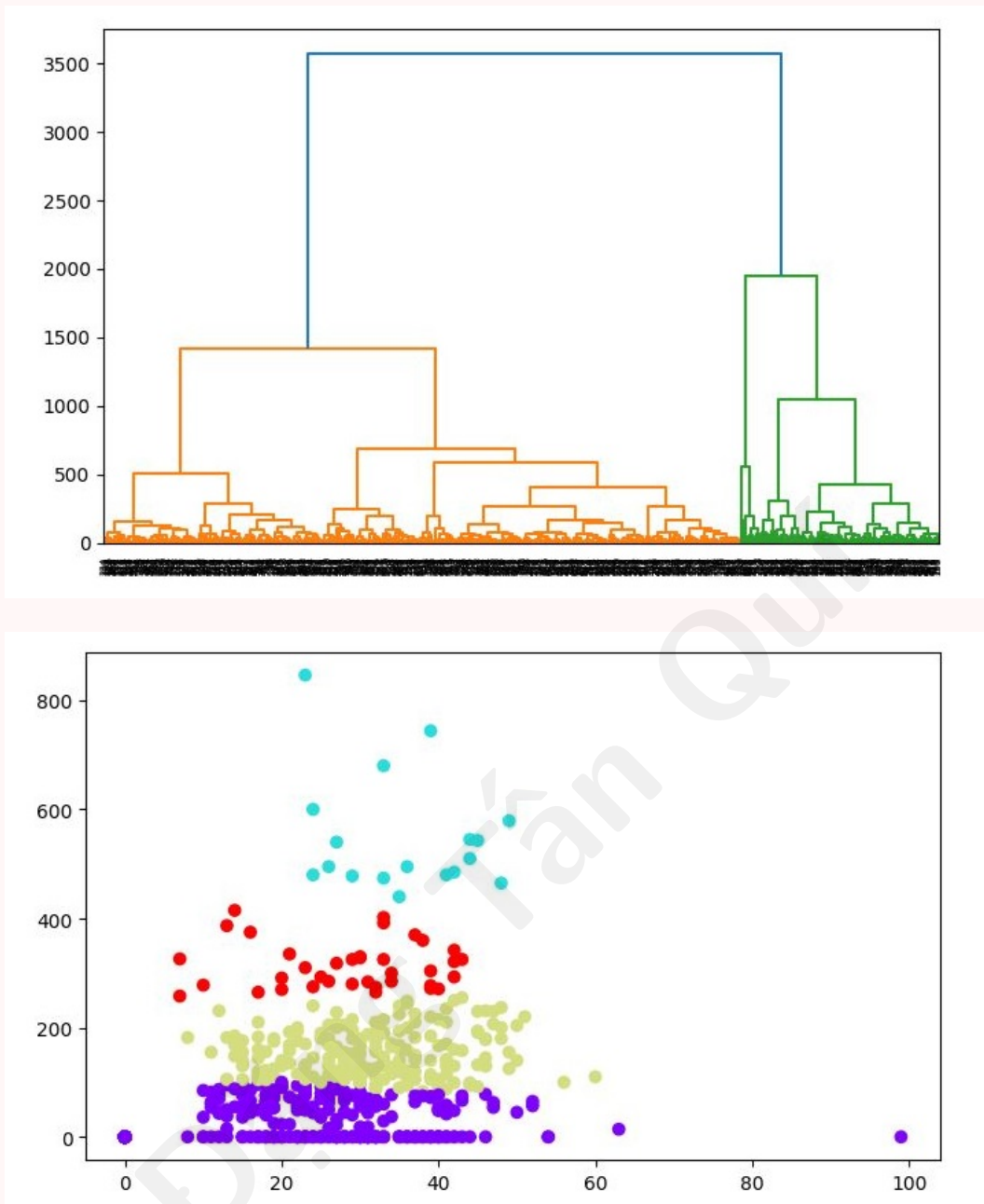
plt.scatter(X[:, 0], X[:, 1], c=cluster.labels_, cmap='rainbow')
```

Hai cụm sau gán nhãn



Minh họa bổ sung

Các hình img05, img06 minh họa thêm các bước gộp cụm và kết quả phân tách trên ví dụ tương tự.



6.2.2 Ví dụ — Cụm trên dữ liệu y khoa

Dataset tiểu đường

Ví dụ minh họa dùng `sklearn.datasets.load_diabetes` (nhiều feature, bối cảnh y khoa). Có thể thay bằng `fetch_openml(name='diabetes', version=1)` nếu cần bộ Pima 8 cột.

Đường dẫn CSV cục bộ

Nếu có file Pima Indians Diabetes cục bộ, đặt đường dẫn hợp lệ và đọc bằng `pandas.read_csv`; không bắt buộc khi đã dùng `load_diabetes`.

Nạp dữ liệu và chọn hai feature

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import load_diabetes
import scipy.cluster.hierarchy as shc
from sklearn.cluster import AgglomerativeClustering

data = load_diabetes()
X_full = data.data
patient_data = X_full[:, 3:5] # hai ct feature (tng t chn ct 3:5)
```

Dendrogram trên toàn bộ feature

```
plt.figure(figsize=(10, 7))
plt.title("Patient Dendograms")
dend = shc.dendrogram(shc.linkage(X_full, method='ward'))
```

Phân 4 cụm trên hai chiều đã chọn

```
cluster = AgglomerativeClustering(n_clusters=4, linkage='ward')
cluster.fit_predict(patient_data)

plt.figure(figsize=(7.2, 5.5))
plt.scatter(patient_data[:, 0], patient_data[:, 1],
            c=cluster.labels_, cmap='rainbow')
plt.show()
```

Hai đồ thị đầu ra

Chạy mã trên sinh dendrogram (toàn feature) và scatter hai chiều tô màu theo 4 cụm — tương ứng hai hình minh họa trong bài (dendrogram bệnh nhân và phân cụm 2D).

Linkage

single, complete, average, ward ảnh hưởng cách đo khoảng cách giữa cụm; ward thường cho cụm compact khi feature đồng thang.

Chi phí

Phân cụm phân cấp tốn $O(n^2)$ – $O(n^3)$ bộ nhớ/thời gian; dataset rất lớn nên cân nhắc K-means hoặc BIRCH.

7 Phân cụm dựa trên mật độ (Density-Based Clustering)

Ý tưởng phân cụm theo mật độ

Phân cụm dựa trên mật độ cho rằng cụm là các vùng **mật độ cao**, ngăn cách bởi vùng **mật độ thấp**.

Ba loại điểm

1. **Core (lõi)**: điểm có ít nhất một số láng giềng tối thiểu trong bán kính cho trước — tâm vùng cụm.
2. **Border (biên)**: không phải core nhưng nằm trong lân cận của ít nhất một điểm core.
3. **Noise (nhiều)**: không phải core cũng không phải border.

So với centroid/partition

Không giả định cụm hình cầu hay số cụm cố định; phù hợp hình dạng bất kỳ và phát hiện điểm nhiễu.

7.1 Các thuật toán phân cụm theo mật độ phổ biến

7.1.1 DBSCAN

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) là thuật toán phân cụm theo mật độ phổ biến nhất. Cần hai tham số:

- **minPts**: số láng giềng tối thiểu để một điểm được coi là core.
- **eps**: bán kính (hoặc khoảng cách tối đa) xét láng giềng giữa các điểm core.

Cơ chế tóm tắt

Từ mỗi core chưa thăm, mở rộng cụm bằng cách nối các core và border liên thông trong ϵ ; điểm không đạt điều kiện gán là noise.

7.1.2 OPTICS

OPTICS

OPTICS (Ordering Points To Identify the Clustering Structure) xây **đồ thị reachability** nối mỗi điểm tới láng giềng gần trong ngưỡng khoảng cách; cạnh có trọng số theo khoảng cách. Từ đồ thị này, thuật toán tạo cấu trúc phân cấp bằng cách tách theo ngưỡng mật độ.

So với DBSCAN

OPTICS xử lý tốt hơn khi cụm có **mật độ khác nhau**, có nhiều, và khi cần xem cấu trúc phân cấp mật độ mà không cố định một cặp **eps/minPts** duy nhất.

7.1.3 HDBSCAN**HDBSCAN**

HDBSCAN (Hierarchical Density-Based Spatial Clustering of Applications with Noise) mở rộng DBSCAN theo hướng phân cấp mật độ.

Ưu điểm so với DBSCAN

- Cụm **mật độ thay đổi** trong cùng dataset.
- Phát hiện cụm **hình dạng và kích thước đa dạng**.
- Ít phụ thuộc chọn một ε duy nhất cho mọi mức mật độ.

Các chương tiếp theo

Ba chương kế tiếp trình bày chi tiết DBSCAN, OPTICS và HDBSCAN kèm triển khai Python.

8 DBSCAN Clustering**DBSCAN**

Density-Based Spatial Clustering of Applications with Noise: thuật toán phân cụm dựa trên mật độ, cần hai tham số — **minPts** (số láng giềng tối thiểu) và **eps** (khoảng cách tối đa giữa điểm lõi).

8.1 Cách DBSCAN hoạt động**Các bước thuật toán**

1. Chọn ngẫu nhiên một điểm dữ liệu chưa được thăm.
2. Nếu điểm có ít nhất **minPts** láng giềng trong bán kính **eps**: tạo cụm mới, thêm điểm và các láng giềng vào cụm.
3. Nếu không đủ láng giềng: đánh dấu điểm là **nhieũ** (noise) và chuyển sang điểm tiếp theo.
4. Lặp các bước 1–3 cho đến khi mọi điểm đã được thăm.

8.2 Triển khai Python (scikit-learn)

Bước 1: Nạp dataset make_moons

```
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=200, noise=0.05, random_state=0)
```

Bước 2: DBSCAN với eps và min_samples

```
from sklearn.cluster import DBSCAN
clustering = DBSCAN(eps=0.2, min_samples=5)
clustering.fit(X)
```

Bước 3: Trực quan hóa kết quả

```
import matplotlib.pyplot as plt
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=clustering.labels_, cmap='rainbow')
plt.show()
```

Ví dụ đầy đủ

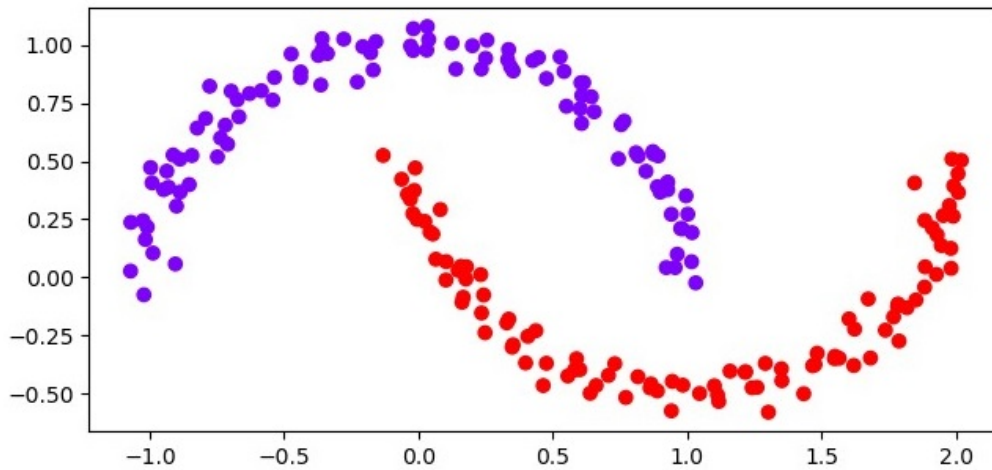
```
from sklearn.datasets import make_moons
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

X, y = make_moons(n_samples=200, noise=0.05, random_state=0)
clustering = DBSCAN(eps=0.2, min_samples=5)
clustering.fit(X)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=clustering.labels_, cmap='rainbow')
plt.show()
```

Kết quả mong đợi

Biểu đồ phân tán cho thấy hai cụm tách bạch (hai “vàng trắng”); điểm nhiễu thường được tô màu đen (nhãn -1).



8.3 Ưu và nhược điểm

Ưu điểm

- Xử lý cụm **hình dạng tùy ý**, không giả định cụm hình cầu như K-means.
- **Không cần biết trước số cụm**.
- **Phát hiện outlier**: điểm xa vùng mật độ cao được coi là nhiễu.
- Tương đối **ít nhạy** với khởi tạo so với K-means.
- **Mở rộng** tốt: chỉ cần khoảng cách giữa các láng giềng, không phải mọi cặp điểm.

Nhược điểm

- **Nhạy** với `eps` và `min_samples`; chọn sai có thể gộp cụm hoặc bỏ sót cụm.
- Kém trên dataset **mật độ không đồng nhất** (giả định mật độ cụm tương tự).
- Có thể **không xác định** giữa các lần chạy.
- **Tốn kém** trên dữ liệu chiều cao (tính khoảng cách đắt).
- Nhiễu/outlier mật độ cao có thể bị gán nhầm vào cụm.

9 OPTICS Clustering

OPTICS

Ordering Points To Identify the Clustering Structure: thuật toán phân cụm dựa trên mật độ, liên quan DBSCAN nhưng mạnh hơn trong nhiều tình huống thực tế.

Ưu thế so với DBSCAN

- Nhận diện cụm **mật độ thay đổi**.
- Xử lý **nhiều** tốt hơn.

- Sinh cấu trúc phân cấp cụm (phân tích ở nhiều mức độ chi tiết).

9.1 Triển khai Python (scikit-learn)

Ví dụ OPTICS với make_blobs

```
from sklearn.cluster import OPTICS
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

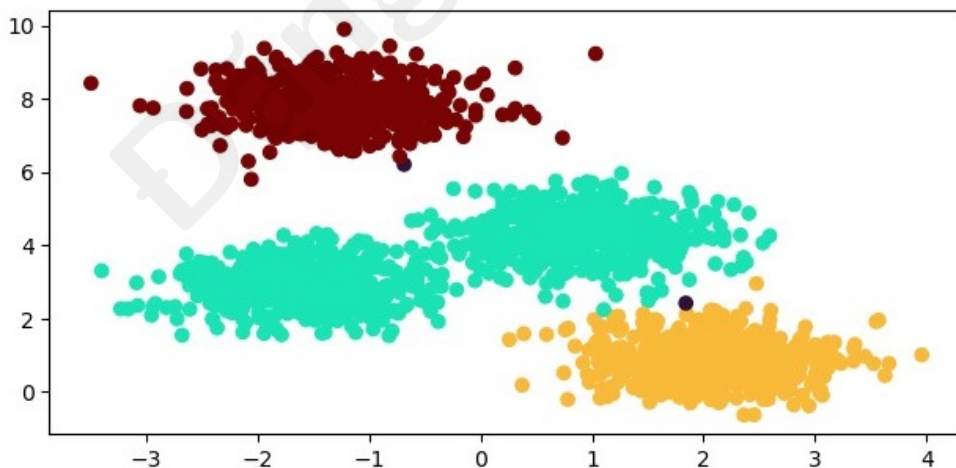
X, y = make_blobs(n_samples=2000, centers=4, cluster_std=0.60,
                  random_state=0)

optics = OPTICS(min_samples=50, xi=.05)
optics.fit(X)

labels = optics.labels_
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='turbo')
plt.show()
```

Tham số

min_samples: số mẫu tối thiểu để hình thành cụm. **xi:** độ dốc của hệ phân cấp cụm (steepness). Sau fit, mỗi điểm có nhãn cụm; vẽ scatter tô màu theo labels_.



9.2 Ưu và nhược điểm

Ưu điểm

- Cụm mật độ không đồng nhất.
- Tách **điểm nhiễu** không thuộc cụm nào.
- **Dendrogram / reachability plot** hỗ trợ phân tích đa mức.

Nhược điểm

- Cần **tinh chỉnh** `min_samples`, `xi` cẩn thận.
- **Phức tạp tính toán** cao trên dataset lớn, đặc biệt khi `min_samples` lớn.

10 HDBSCAN Clustering

HDBSCAN

Hierarchical Density-Based Spatial Clustering of Applications with Noise: mở rộng DBSCAN, xử lý cụm mật độ và hình dạng đa dạng tốt hơn.

10.1 Cách HDBSCAN hoạt động

Mutual reachability và phân cấp

1. Xây đồ thị **mutual-reachability**: mỗi điểm là nút, cạnh có trọng số theo độ tương đồng/khoảng cách; nối hai điểm nếu **khoảng cách mutual reachability** dưới ngưỡng.
2. **Mutual reachability** giữa hai điểm là max của hai reachability distance; reachability phản ánh mức “dễ” đi từ điểm này sang điểm kia (phụ thuộc mật độ trên đường đi).
3. Trích **phân cấp cụm** từ đồ thị bằng thuật toán **minimum spanning tree (MST)**: lá = điểm đơn lẻ, nút trong = cụm nhiều kích thước/hình dạng.
4. Áp dụng **condensed tree** lên MST, cắt ở mức phù hợp (kích thước cụm tối thiểu hoặc độ ổn định cụm) để ra cụm cuối.

10.2 Cài đặt và sử dụng (gói hdbscan)

Cài đặt pip

```
pip install hdbscan
```

Huấn luyện trên make_blobs

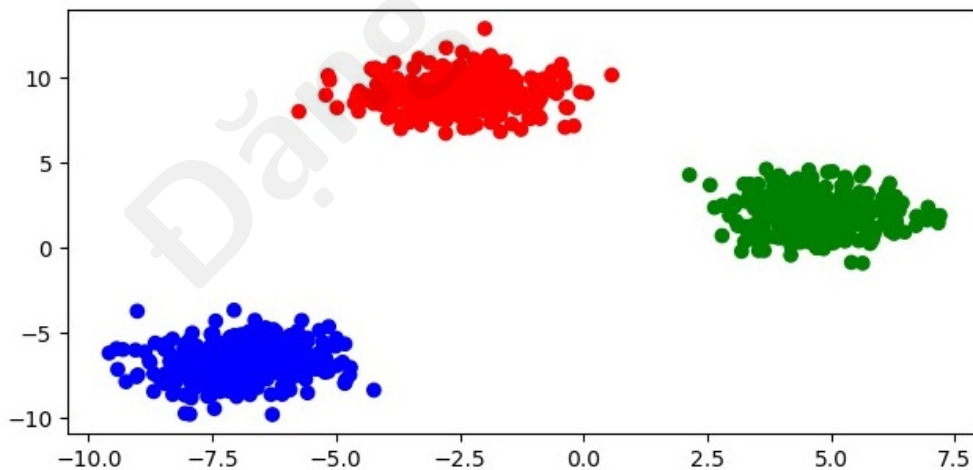
```
import hdbscan
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=1000, centers=3, random_state=42)

clusterer = hdbscan.HDBSCAN(min_cluster_size=10, metric='euclidean')
clusterer.fit(X)

labels = clusterer.labels_
colors = np.array([x for x in 'bgrcmykbgrcmykbgrcmykbgrcmyk'])
colors = np.hstack([colors] * 20)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=colors[labels])
plt.show()
```



10.3 Tham số chính

Tham số điều chỉnh

- `min_cluster_size`: kích thước cụm tối thiểu; điểm không thuộc cụm nào được gán `noise`.
- `min_samples`: số mẫu tối thiểu trong lân cận để điểm là **core**.

- `cluster_selection_epsilon`: bán kính lân cận khi chọn cụm.
- `metric`: metric khoảng cách (ví dụ `euclidean`).

Ưu điểm

- Cụm mật độ khác nhau.
- Cụm không hình cầu, kích thước đa dạng.
- Không cần chỉ định số cụm.
- Chống nhiễu: outlier gán nhãn noise.

11 BIRCH Clustering

BIRCH

Balanced **I**terative **R**educing and **C**lustering **H**ierarchies: thuật toán phân cụm phân cấp cho **dataset lớn**; xây cây cụm bằng chia đệ quy đến khi đạt tiêu chí dừng.

CF và SCF

- **Clustering Feature (CF)**: tóm tắt thống kê một tập điểm.
- **Sub-Cluster Feature (SCF)**: biểu diễn cấu trúc các subcluster.

11.1 Ba bước chính

Ba giai đoạn BIRCH

1. **Khởi tạo**: tạo cây rỗng; đặt số CF tối đa mỗi nút.
2. **Phân cụm**: đọc từng điểm; cập nhật CF nếu có, hoặc tạo CF mới; nếu vượt ngưỡng CF trong nút thì **chia subcluster** đệ quy.
3. **Tinh chỉnh (Refinement)**: gộp subcluster tương tự theo metric khoảng cách.

11.2 Triển khai Python (scikit-learn)

Ví dụ Birch

```
from sklearn.datasets import make_blobs
from sklearn.cluster import Birch
import matplotlib.pyplot as plt

X, y = make_blobs(n_samples=1000, centers=10, cluster_std=0.50,
                  random_state=0)
```

```

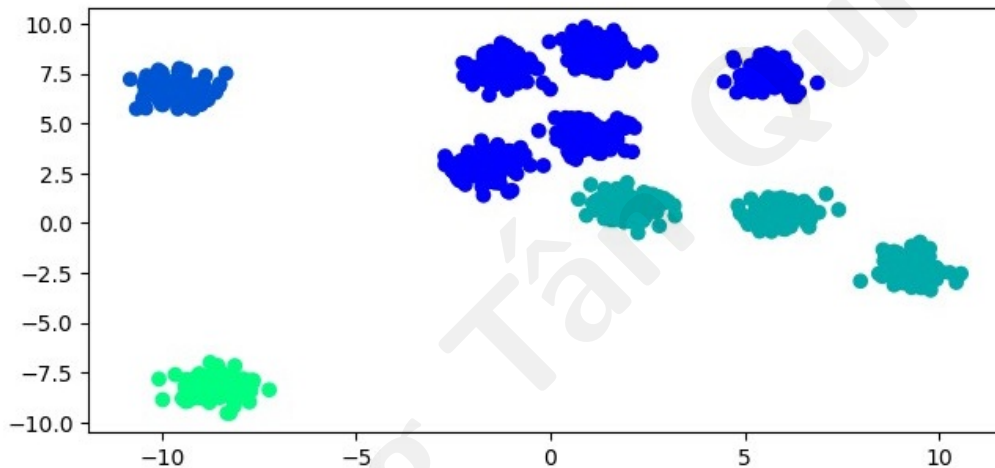
birch = Birch(threshold=1.5, n_clusters=4)
birch.fit(X)
labels = birch.predict(X)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='winter')
plt.show()

```

Tham số

threshold: ngưỡng tạo subcluster. **n_clusters**: số cụm đầu ra sau tinh chỉnh. Dùng **fit** rồi **predict** để lấy nhãn.



11.3 Ưu và nhược điểm

Ưu điểm

- **Khả mở rộng** trên dữ liệu lớn nhờ cấu trúc cây.
- **Tiết kiệm bộ nhớ** nhờ CF/SCF.
- **Nhanh** nhờ phân cụm gia tăng (incremental).

Nhược điểm

- **Nhạy** tham số (số CF tối đa mỗi nút, ngưỡng subcluster).
- Giả định cụm **gần hình cầu**; kém với cụm phi cầu.
- Mặc định dùng **Euclidean**; có thể không phù hợp mọi bài toán.

12 Affinity Propagation

Affinity Propagation

Thuật toán xác định các **exemplar** (điểm đại diện) và gán mỗi mẫu vào một exemplar. **Không cần** chỉ định trước số cụm — hữu ích cho phân tích khám phá (Frey & Dueck, 2007).

12.1 Responsibility và Availability

Hai ma trận lặp

- **Responsibility** $R(i, k)$: mức phù hợp của điểm k làm exemplar cho điểm i .
- **Availability** $A(i, k)$: mức điểm i “muốn” chọn k làm exemplar.

Thuật toán **luân phiên** cập nhật hai ma trận đến hội tụ; exemplar cuối chọn theo giá trị lớn nhất trong ma trận responsibility.

12.2 Triển khai Python (scikit-learn)

Ví dụ AffinityPropagation

```
from sklearn.cluster import AffinityPropagation
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=100, centers=4, random_state=0)

af = AffinityPropagation(preference=-50)
af.fit(X)

print("Cluster labels:", af.labels_)
print("Exemplars:", af.cluster_centers_indices_)

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=af.labels_, cmap='viridis')
plt.scatter(af.cluster_centers_[0], af.cluster_centers_[1],
            marker='x', color='red')
plt.show()
```

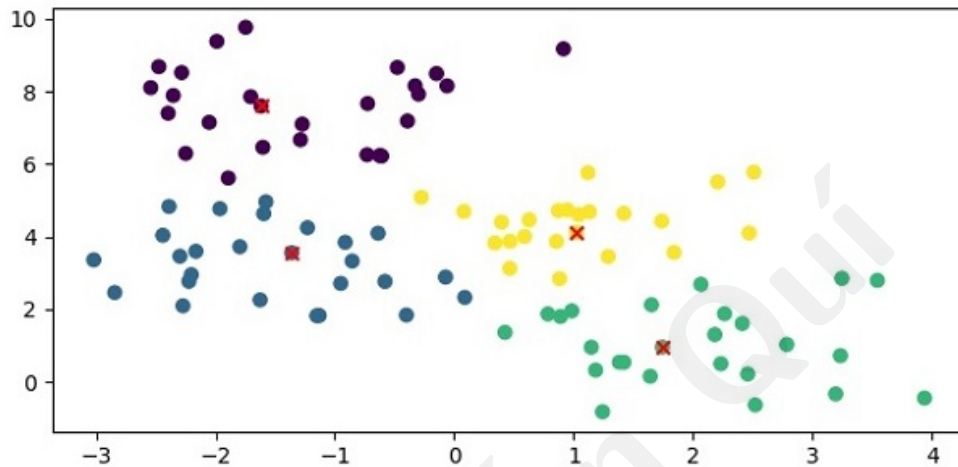
Output mẫu trên terminal

```
Cluster labels: [3 0 3 3 3 3 1 0 0 0 0 0 0 0 0 2 3 3 1 2 2 0 1 2 3 1 3 3 2 2
 2 0 2 2 1 3 0 2 0 1 3 1 0 1 1 0 2 1 3 1 3 2 1 1 1 0 0 2 2 0 0 2 2 3 2 0 1 1 2
 3 0 2 3 0 3 3 3 1 2 2 2 0 1 1 2 1 2 2 3 3 3 1 1 1 1 0 0 1 0 1]
```

Exemplars: [9 41 51 74]

Preference và Damping

- **Preference:** càng cao càng nhiều exemplar; càng thấp càng ít cụm.
- **Damping:** hệ số làm mượt hội tụ; damping lớn $\Rightarrow$ hội tụ chậm hơn.



12.3 Ưu và nhược điểm

Ưu điểm

- Tự xác định số cụm.
- Cụm hình dạng/kích thước tùy ý.
- Chịu được dữ liệu **nhiều** hoặc **thiếu**.
- Tương đối **ổn định** với khởi tạo.

Nhược điểm

- **Tốn tài nguyên** trên dataset lớn hoặc nhiều feature.
- Có thể hội tụ **chưa tối ưu** khi dữ liệu biến thiên/nhiều cao.
- Nhạy damping.
- Có thể sinh nhiều cụm **quá nhỏ** khó diễn giải.

13 Phân cụm dựa trên phân phối (Distribution-Based)

Phân cụm dựa trên phân phối

Còn gọi **phân cụm xác suất**: giả định dữ liệu sinh từ **hỗn hợp** phân phối xác suất; ước lượng phân phối nền rồi gom điểm có tính chất tương tự.

13.1 Gaussian Mixture Model (GMM)

GMM

Giả định dữ liệu từ hỗn hợp phân phối **Gaussian**; mỗi Gaussian là một cụm. Ưu điểm: cụm **chồng lấn**, mô hình hóa **hiệp phương sai**, gán cụm **xác suất** cho từng điểm. Ứng dụng: phân đoạn ảnh, nhận dạng mẫu, phát hiện bất thường.

13.2 Triển khai Python (GaussianMixture, covariance full)

Ví dụ GMM

```
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs
import matplotlib.pyplot as plt

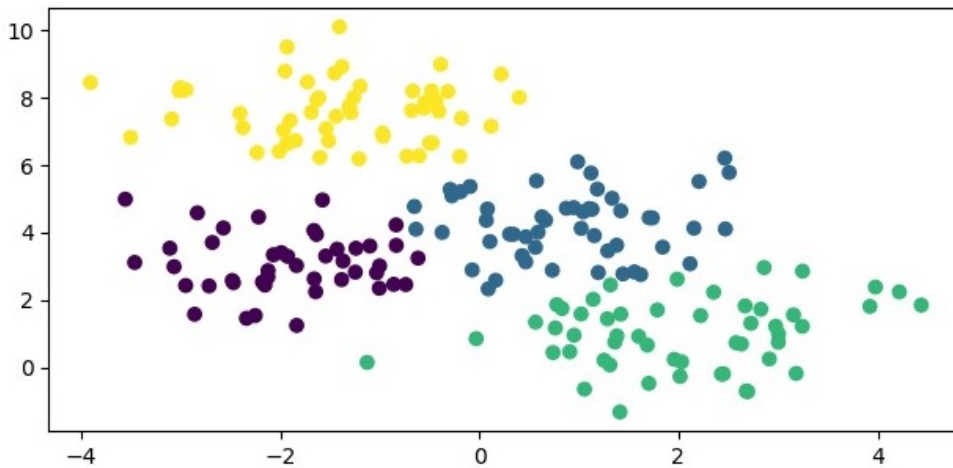
X, _ = make_blobs(n_samples=200, centers=4, random_state=0)

gmm = GaussianMixture(n_components=4, covariance_type='full')
gmm.fit(X)
labels = gmm.predict(X)

print("Cluster labels:", labels)
plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.show()
```

covariance_type

full: ma trận hiệp phương sai đầy đủ mỗi cụm. Các lựa chọn khác: tied, diag, spherical. Tham số khởi tạo điều khiển cách ước lượng tham số Gaussian ban đầu.



13.3 Ưu và nhược điểm GMM

Ưu điểm

- Mô hình hóa phân phối **linh hoạt**.
- Xử lý dữ liệu **thiếu**.
- Khung **xác suất** (độ không chắc chắn gắn cụm).
- Ước lượng mật độ và **sinh mẫu** mới.
- Dùng trong **bán giám sát** khi một phần có nhãn.

Nhược điểm

- **Nhạy** số cụm và khởi tạo mean/covariance.
- **Đắt** trên chiều cao (nghịch đảo ma trận hiệp phương sai).
- Giả định **Gaussian** có thể không đúng với mọi dữ liệu.
- Dễ **overfitting** khi nhiều tham số hoặc ít mẫu.
- Khó diễn giải khi ma trận hiệp phương sai phức tạp.

14 Agglomerative Clustering

Agglomerative Clustering

Thuật toán phân cụm **phân cấp bottom-up**: mỗi điểm ban đầu là một cụm, lặp gộp hai cụm gần nhất đến khi đạt tiêu chí dừng. Sinh **dendrogram** thể hiện quan hệ phân cấp giữa các cụm.

14.1 Các bước thuật toán

Các bước

1. Coi **mỗi điểm** là một cụm riêng.
2. Tính **ma trận gần** (proximity) bằng metric khoảng cách.
3. **Gộp** hai cụm theo tiêu chí linkage (single, complete, average, ward, ...).
4. **Cập nhật** ma trận khoảng cách.
5. Lặp bước 3–4 cho đến khi còn **một cụm** (hoặc dừng sớm theo ngưỡng).

Vì sao dùng Agglomerative?

Dễ diễn giải quan hệ giữa điểm; không bắt buộc biết số cụm trước khi xây dendrogram (số cụm chọn khi “cắt” cây). Hiệu quả, phát hiện được cụm nhỏ.

14.2 Triển khai Python (Iris, dendrogram)

Nạp Iris và linkage ward

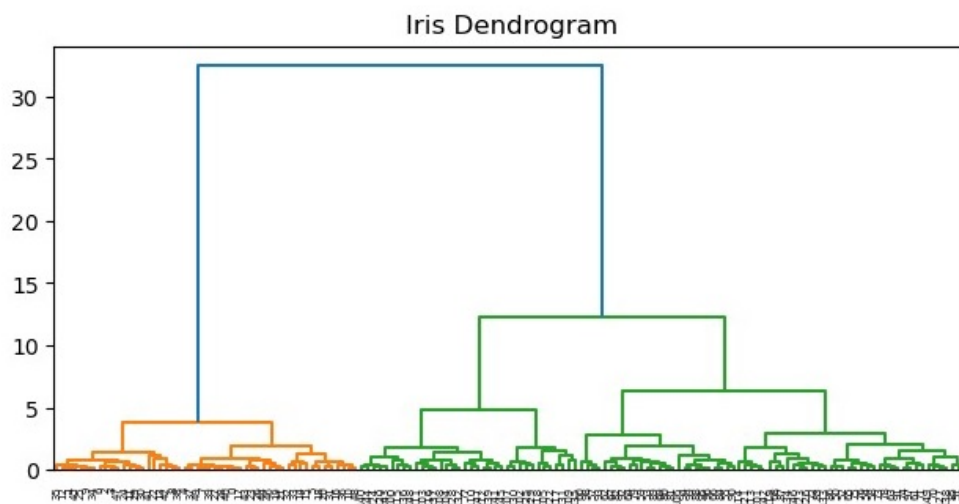
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

iris = load_iris()
X = iris.data
y = iris.target

Z = linkage(X, 'ward')
```

Vẽ dendrogram

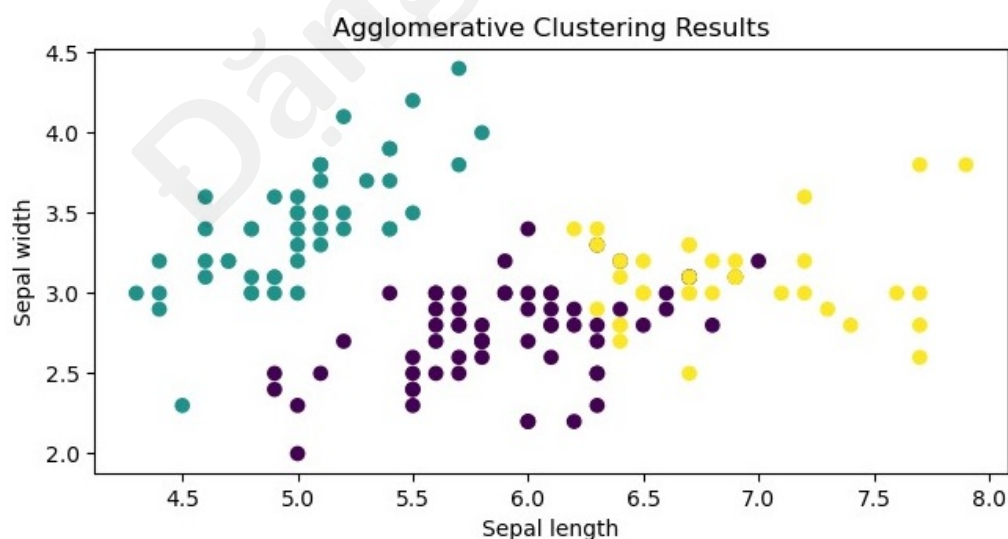
```
plt.figure(figsize=(7.5, 3.5))
plt.title("Iris Dendrogram")
dendrogram(Z)
plt.show()
```



AgglomerativeClustering và scatter kết quả

```
model = AgglomerativeClustering(n_clusters=3)
model.fit(X)
labels = model.labels_

plt.figure(figsize=(7.5, 3.5))
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")
plt.title("Agglomerative Clustering Results")
plt.show()
```



ward

Phương pháp **ward** tối thiểu hóa phương sai khoảng cách khi gộp cụm — thường cho cụm cân bằng hơn trên Iris.

14.3 Ưu, nhược điểm và ứng dụng

Ưu điểm

- **Dendrogram** thể hiện quan hệ phân cấp.
- Nhiều **metric** và **linkage**.
- Linh hoạt **số cụm** khi cắt cây.
- Triển khai hiệu quả cho dataset lớn vừa phải.

Nhược điểm

- **Tốn kém** trên dataset rất lớn.
- Cụm **mất cân bằng** nếu metric/linkage không phù hợp.
- Kết quả **nhạy** lựa chọn metric và linkage.
- Dendrogram **khó đọc** khi có rất nhiều cụm/điểm.

Ứng dụng

- **Phân đoạn ảnh** (Image Segmentation).
- **Phân cụm tài liệu** (Document Clustering).
- **Phân khúc khách hàng** / hành vi khách hàng.
- **Phân khúc thị trường** (Market Segmentation).
- **Phân tích mạng xã hội**.